

PoRE Lab3 实验报告

姓名：徐锦云 学号：23307130447 耗时：12小时

PoRE Lab3 实验报告

Task 1: 简单汇编代码理解

Task1.1 max函数实现代码

分析思路

Task1.2 func1函数还原

分析思路

task1_value

Task 2: 复杂汇编代码解释与输入输出推测

func2的功能是什么?

尝试还原代码

经过func2后, str的值为?

Task 3: C++汇编代码理解与输出推测

推测task3_value的值

分析思路

尝试还原整个代码

值

解释该汇编代码中max、min的实现上与Task1的差异

max、min的源码

实验所有运行截图

Task1&Task2

Task3

Task 1: 简单汇编代码理解

Task1.1 max函数实现代码

```
int max(int x, int y) {
    int res = y;
    if (x > y) {
        res = x;
    }
    return res;
}
```

分析思路

```
max:
.LFB0:
.cfi_startproc
sub sp, sp, #32
.cfi_def_cfa_offset 32
str w0, [sp, 12]
str w1, [sp, 8]
ldr w0, [sp, 8]
str w0, [sp, 28]
```

```

    ldr w1, [sp, 12]
    ldr w0, [sp, 8]
    cmp w1, w0
    ble .L2
    ldr w0, [sp, 12]
    str w0, [sp, 28]
.L2:
    ldr w0, [sp, 28]
    add sp, sp, 32
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
.LFE0:
    .size    max, .-max
    .align   2
    .global func1
    .type    func1, %function

```

以上就是max函数的全部内容。首先分配32字节的栈空间，随后将寄存器w0中的值放到栈偏移12字节处，将寄存器w1中的值放到栈偏移8字节处，紧接着加载原先存放w1的地址处的值存入寄存器w0中（从内存地址 一个 32 位的值，放到寄存器 sp + 8 处读取 w0 中），随后将寄存器w0中的值放到栈偏移28字节处，紧接着加载原先存放w0的地址处的值存入寄存器w1中，再从内存地址 sp + 8 处读取一个 32 位的值，放到寄存器 w0 中。此时，比较w1和w0的大小，如果w1<=w0，则取w0的值，否则取w1的值。

Task1.2 func1函数还原

```

int func1(int n) {
    // 递归基: 当 n <= 1 时直接返回 1
    if (n <= 1) {
        return 1;
    }
    // 根据 n 的奇偶性分支
    if ((n & 1) == 0) {
        // n 为偶数: 直接递归到 n-1, 不额外累加当前 n
        return func1(n - 1);
    } else {
        // n 为奇数: 在 func1(n-1) 的结果基础上再加上当前 n
        return func1(n - 1) + n;
    }
}

```

分析思路

```

func1:
.LFB1:
    .cfi_startproc
    stp x29, x30, [sp, -32]!
    .cfi_def_cfa_offset 32
    .cfi_offset 29, -32
    .cfi_offset 30, -24
    mov x29, sp

```

```

    str w0, [sp, 28]
    ldr w0, [sp, 28]
    cmp w0, 1
    bgt .L5
    mov w0, 1
    b .L6
.L5:
    ldr w0, [sp, 28]
    and w0, w0, 1
    cmp w0, 0
    bne .L7
    ldr w0, [sp, 28]
    sub w0, w0, #1
    bl func1
    b .L6
.L7:
    ldr w0, [sp, 28]
    sub w0, w0, #1
    bl func1
    mov w1, w0
    ldr w0, [sp, 28]
    add w0, w1, w0
.L6:
    ldp x29, x30, [sp], 32
    .cfi_restore 30
    .cfi_restore 29
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
.LFE1:
    .size func1, .-func1
    .align 2
    .global func2
    .type func2, %function

```

首先，`func1` 将参数 `n`（寄存器 `w0`）保存到栈上偏移 28 字节的位置 `[sp, 28]`。随后：

- 通过 `cmp w0, 1` 和 `bgt .L5` 判断 `n` 是否大于 1：
 - 若 `n <= 1`，进入“递归结束”分支：`mov w0, 1; b .L6;`，直接返回常数 1；
 - 若 `n > 1`，则跳转到 `.L5` 继续分析。
- 在 `.L5` 中，再次取回 `n`，执行 `and w0, w0, 1; cmp w0, 0; bne .L7:`
 - 这一段通过按位与 1 判断 `n` 的奇偶性：
 - `n & 1 == 0`（偶数）不会跳到 `.L7`；
 - `n & 1 != 0`（奇数）会跳到 `.L7`。

当 `n` 为偶数 时（不跳转到 `.L7`）：

- 重新从栈中取回原始的 `n`，`sub w0, w0, #1` 得到 `n-1`，然后 `bl func1` 调用自身，计算 `func1(n-1)`；
- 随后直接跳转到 `.L6` 返回，不再对返回值做任何额外运算。

这一段正好对应 C 代码中的：

```
if ((n & 1) == 0) {  
    return func1(n - 1);  
}
```

当 **n 为奇数** 时（跳转到 `.L7`）：

- 同样先将 `n-1` 放入 `w0`，并递归调用 `func1(n-1)`；
- 函数返回后，`w0` 中保存的是 `func1(n-1)` 的值，接着执行：

```
mov w1, w0      // w1 = func1(n-1)  
ldr w0, [sp, 28] // 重新加载原始的 n  
add w0, w1, w0   // w0 = func1(n-1) + n
```

这与源码里的：

```
return func1(n - 1) + n;
```

完全一致。

综合两种情况，可以看出 `func1` 的语义是：

- 以 1 作为递归基础；
- 对于所有不大于 `n` 的奇数，将它们累加起来：
 - 偶数只“向下传递”到 `n-1`；
 - 奇数则在递归结果上额外加上当前奇数。

因此，对于本实验中 `random_key = max(18, 16) = 18`，有：

$$func1(18) = 1 + 3 + 5 + 7 + 9 + 11 + 13 + 15 + 17 = 81$$

task1_value

根据：

```
,  
int lab3Task1_23307130447() {  
    int random_key = max(18, 16);  
    int task1_value = func1(random_key);  
    return task1_value;  
}
```

因此： `task1_value = 81`

Task 2: 复杂汇编代码解释与输入输出推测

func2的功能是什么？

```
func2:
.LFB2:
    .cfi_startproc
    sub sp, sp, #32
    .cfi_def_cfa_offset 32
    str x0, [sp, 8]
    str wzr, [sp, 20]
    b    .L9
.L10:
    ldr w0, [sp, 20]
    add w0, w0, 1
    str w0, [sp, 20]
.L9:
    ldrsw  x0, [sp, 20]
    ldr x1, [sp, 8]
    add x0, x1, x0
    ldrb   w0, [x0]
    cmp w0, 0
    bne .L10
    str wzr, [sp, 24]
    b    .L11
.L15:
    str wzr, [sp, 28]
    b    .L12
.L14:
    ldrsw  x0, [sp, 28]
    ldr x1, [sp, 8]
    add x0, x1, x0
    ldrb   w1, [x0]
    ldrsw  x0, [sp, 28]
    add x0, x0, 1
    ldr x2, [sp, 8]
    add x0, x2, x0
    ldrb   w0, [x0]
    cmp w1, w0
    bls .L13
    ldrsw  x0, [sp, 28]
    ldr x1, [sp, 8]
    add x0, x1, x0
    ldrb   w0, [x0]
    strb   w0, [sp, 19]
    ldrsw  x0, [sp, 28]
    add x0, x0, 1
    ldr x1, [sp, 8]
    add x1, x1, x0
    ldrsw  x0, [sp, 28]
    ldr x2, [sp, 8]
    add x0, x2, x0
```

```

        ldrb    w1, [x1]
        strb    w1, [x0]
        ldrsw   x0, [sp, 28]
        add     x0, x0, 1
        ldr     x1, [sp, 8]
        add     x0, x1, x0
        ldrb    w1, [sp, 19]
        strb    w1, [x0]
.L13:
        ldr     w0, [sp, 28]
        add     w0, w0, 1
        str     w0, [sp, 28]
.L12:
        ldr     w1, [sp, 20]
        ldr     w0, [sp, 24]
        sub     w0, w1, w0
        sub     w0, w0, #1
        ldr     w1, [sp, 28]
        cmp     w1, w0
        blt     .L14
        ldr     w0, [sp, 24]
        add     w0, w0, 1
        str     w0, [sp, 24]
.L11:
        ldr     w0, [sp, 20]
        sub     w0, w0, #1
        ldr     w1, [sp, 24]
        cmp     w1, w0
        blt     .L15
        nop
        nop
        add     sp, sp, 32
        .cfi_def_cfa_offset 0
        ret
        .cfi_endproc
.LFE2:
        .size   func2, .-func2
        .align  2
        .global lab3Task1_23307130447
        .type   lab3Task1_23307130447, %function
;

```

结合汇编代码，可以更精细地对应每一个局部变量与循环结构：

- `[sp, 8]`：保存传入的字符串首地址 `str`；
- `[sp, 20]`：变量 `len`，用来统计字符串长度；
- `[sp, 24]`：外层循环变量 `i`；
- `[sp, 28]`：内层循环变量 `j`；
- `[sp, 19]`：交换字符时的临时变量，对应 C 代码中的 `tmp`。

第一个 while 循环（计算长度）：

- `str wzr, [sp, 20]` 把 `len` 清零;
- 标签 `.L9 / .L10` 之间, 依次根据 `[sp, 20]` 作为偏移, 用 `ldrs w0, [sp, 20]; add x0, x1, x0; ldrb w0, [x0]` 从 `str[len]` 位置取字符, 若非 0 则在 `.L10` 中对 `len` 自增, 再回到 `.L9`, 直到遇到 `'\0'` 为止。
这是:

```
len = 0;
while (str[len] != '\0') {
    len++;
}
```

外层 for 循环 (i) :

- `str wzr, [sp, 24]` 将 `i` 置为 0;
- 在 `.L11` 中, 通过 `[sp, 24]` 与 `[sp, 20]` 的运算, 构造出条件 `i < len - 1`, 对应 `for (i = 0; i < len - 1; ++i)`;
- 每次内层循环结束后, `ldr w0, [sp, 24]; add w0, w0, 1; str w0, [sp, 24]` 实现 `i++`。

内层 for 循环 (j) 和字符交换:

- `str wzr, [sp, 28]` 初始化 `j = 0`, 随后在 `.L12`、`.L14` 附近通过 `[sp, 28]` 和 `[sp, 20]`、`[sp, 24]` 的组合比较实现 `j < len - i - 1` 的条件;
- 在 `.L14` 中, 利用偏移 `j` 和 `j+1` 分别读取 `str[j]` 和 `str[j+1]`, `cmp w1, w0; bls .L13` 对应 `if (str[j] > str[j+1])`;
- 若需要交换, 则:
 - 先把 `str[j]` 写入 `[sp, 19]`;
 - 再把 `str[j+1]` 写到 `str[j]` 的位置;
 - 最后把 `[sp, 19]` 中保存的字符写回 `str[j+1]`。

因此可以确定, `func2` 的整体功能就是: **对输入字符串按字符的 ASCII 码值进行升序排序。**

尝试还原代码

```
void func2(char *str) {
    // 先计算字符串长度 len
    int len = 0;
    while (str[len] != '\0') {
        len++;
    }

    // 两层循环的冒泡排序: 外层控制“已排好”的前缀边界
    for (int i = 0; i < len - 1; ++i) {
        // 内层从 i 开始, 对后面的字符两两比较并交换
        for (int j = i; j < len - 1; ++j) {
            if (str[j] > str[j + 1]) {
                char tmp = str[j];
                str[j] = str[j + 1];
                str[j + 1] = tmp;
            }
        }
    }
}
```

```

    }
}
}
}

```

经过func2后，str的值为？

根据：

```

int lab3Task1_23307130447() {
    int random_key = max(18, 16);
    int task1_value = func1(random_key);
    return task1_value;
}

char* lab3Task2_23307130447(char* str) {
    func2(str);
    return str;
}

int main(int argc, const char** argv) {
    char task2_value[] = "23307130447,0The1quick2brown3fox4jumps5over6the7lazy8dog9.";

    printf("ID=23307130447 Lab3 Task1 = %d\n", lab3Task1_23307130447());
    printf("ID=23307130447 Lab3 Task2 = %s\n", lab3Task2_23307130447(task2_value));

    return 0;
}

```

原始输入字符串为：

```

175  int main(int argc, const char** argv) {
176      char task2_value[] = "23307130447,0The1quick2brown3fox4jumps5over6the7lazy8dog9.";
177

```

经过ASCII码的升序排列，得到：

```
str=",.000112233334445677789Tabcdeeeefghhijklmnooopqrrstuvwxyz"
```

Task 3: C++汇编代码理解与输出推测

推测task3_value的值

分析思路

```

.arch armv8-a
.file "Lab3_Task3_23307130447.cpp"
.text
#APP
.globl _ZSt21ios_base_library_initv
#NO_APP
.align 2

```



```

.global _Z3maxii
.type _Z3maxii, %function
_Z3maxii:
.LFB1986:
.cfi_startproc
sub sp, sp, #16
.cfi_def_cfa_offset 16
str w0, [sp, 12]
str w1, [sp, 8]
ldr w1, [sp, 12]
ldr w0, [sp, 8]
cmp w1, w0
ble .L2
ldr w0, [sp, 12]
b .L4
.L2:
ldr w0, [sp, 8]
.L4:
add sp, sp, 16
.cfi_def_cfa_offset 0
ret
.cfi_endproc
.LFE1986:
.size _Z3maxii, .-_Z3maxii
.align 2
.global _Z3minRis_
.type _Z3minRis_, %function
_Z3minRis_:
.LFB1987:
.cfi_startproc
sub sp, sp, #16
.cfi_def_cfa_offset 16
str x0, [sp, 8]
str x1, [sp]
ldr x0, [sp, 8]
ldr w1, [x0]
ldr x0, [sp]
ldr w0, [x0]
cmp w1, w0
bge .L6
ldr x0, [sp, 8]
ldr w0, [x0]
b .L8
.L6:
ldr x0, [sp]
ldr w0, [x0]
.L8:
add sp, sp, 16
.cfi_def_cfa_offset 0
ret
.cfi_endproc
.LFE1987:
.size _Z3minRis_, .-_Z3minRis_

```

```

.section    .text._ZN9lab3ClassC2Eii,"axG",@progbits,_ZN9lab3ClassC5Eii,comdat
.align     2
.weak     _ZN9lab3ClassC2Eii
.type     _ZN9lab3ClassC2Eii, %function
_ZN9lab3ClassC2Eii:
.LFB1989:
.cfi_startproc
sub sp, sp, #16
.cfi_def_cfa_offset 16
str x0, [sp, 8]
str w1, [sp, 4]
str w2, [sp]
ldr x0, [sp, 8]
ldr w1, [sp, 4]
str w1, [x0]
ldr x0, [sp, 8]
ldr w1, [sp]
str w1, [x0, 4]
ldr w1, [sp, 4]
ldr w0, [sp]
add w0, w1, w0
lsl w1, w0, 1
ldr x0, [sp, 8]
str w1, [x0, 8]
nop
add sp, sp, 16
.cfi_def_cfa_offset 0
ret
.cfi_endproc
.LFE1989:
.size     _ZN9lab3ClassC2Eii, .-_ZN9lab3ClassC2Eii
.weak     _ZN9lab3ClassC1Eii
.set      _ZN9lab3ClassC1Eii,_ZN9lab3ClassC2Eii
.section   .text._ZN9lab3Class4funcES_, "axG",@progbits,_ZN9lab3Class4funcES_,comdat
.align     2
.weak     _ZN9lab3Class4funcES_
.type     _ZN9lab3Class4funcES_, %function
_ZN9lab3Class4funcES_:
.LFB1991:
.cfi_startproc
sub sp, sp, #80
.cfi_def_cfa_offset 80
stp x29, x30, [sp, 48]
.cfi_offset 29, -32
.cfi_offset 30, -24
add x29, sp, 48
str x19, [sp, 64]
.cfi_offset 19, -16
str x0, [sp, 24]
mov x0, x1
mov x1, x2
str x0, [sp, 8]
ldr w0, [sp, 16]

```

```

bfi w0, w1, 0, 32
str w0, [sp, 16]
adrp x0, :got:__stack_chk_guard
ldr x0, [x0, :got_lo12:__stack_chk_guard]
ldr x1, [x0]
str x1, [sp, 40]
mov x1, 0
add x0, sp, 8
bl _ZN9lab3Class4getMEv
mov w19, w0
add x0, sp, 8
bl _ZN9lab3Class4getNEv
add w0, w19, w0
lsl w0, w0, 1
str w0, [sp, 36]
ldr w2, [sp, 36]
ldr x0, [sp, 24]
ldr w0, [x0, 8]
mov w1, w0
mov w0, w2
bl _Z3maxii
mov w19, w0
ldr x0, [sp, 24]
add x1, x0, 8
add x0, sp, 36
bl _Z3minRiS_
sub w0, w19, w0
mov w1, w0
adrp x0, :got:__stack_chk_guard
ldr x0, [x0, :got_lo12:__stack_chk_guard]
ldr x3, [sp, 40]
ldr x2, [x0]
subs x3, x3, x2
mov x2, 0
beq .L12
bl __stack_chk_fail
.L12:
mov w0, w1
ldr x19, [sp, 64]
ldp x29, x30, [sp, 48]
add sp, sp, 80
.cfi_restore 29
.cfi_restore 30
.cfi_restore 19
.cfi_def_cfa_offset 0
ret
.cfi_endproc
.LFE1991:
.size _ZN9lab3Class4funcES_, .-_ZN9lab3Class4funcES_
.section .text._ZN9lab3Class4getMEv,"axG",@progbits,_ZN9lab3Class4getMEv,comdat
.align 2
.weak _ZN9lab3Class4getMEv
.type _ZN9lab3Class4getMEv, %function

```

```

_ZN9lab3Class4getMEv:
.LFB1992:
    .cfi_startproc
    sub sp, sp, #16
    .cfi_def_cfa_offset 16
    str x0, [sp, 8]
    ldr x0, [sp, 8]
    ldr w0, [x0]
    add sp, sp, 16
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
.LFE1992:
    .size    _ZN9lab3Class4getMEv, .-_ZN9lab3Class4getMEv
    .section .text._ZN9lab3Class4getNEv,"axG",@progbits,_ZN9lab3Class4getNEv,comdat
    .align   2
    .weak    _ZN9lab3Class4getNEv
    .type    _ZN9lab3Class4getNEv, %function
_ZN9lab3Class4getNEv:
.LFB1993:
    .cfi_startproc
    sub sp, sp, #16
    .cfi_def_cfa_offset 16
    str x0, [sp, 8]
    ldr x0, [sp, 8]
    ldr w0, [x0, 4]
    add sp, sp, 16
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
.LFE1993:
    .size    _ZN9lab3Class4getNEv, .-_ZN9lab3Class4getNEv
    .text
    .align   2
    .global  _Z21lab3Task3_23307130447v
    .type    _Z21lab3Task3_23307130447v, %function
;

```

- **函数入口含义：**

`_Z21lab3Task3_23307130447v` 是 C++ 编译器对 `int lab3Task3_23307130447()` 做 name mangling 后得到的名字：前缀 `_Z` 表示 C++ 符号，`21` 表示后面函数名长度为 21，结尾的 `v` 表示“无参数”。这确认了它正是实验手册中 `lab3Task3` 的实现位置。

- **栈帧与局部对象构造：**

在该标签后紧跟标准的函数序言（保存 `x29/x30`，调整 `sp` 等），说明函数会在栈上为局部变量开辟空间。结合后续对 `_ZN9lab3ClassC1Eii` 的两次调用，可以推断：

- 第一次以 `(3,7)` 为参数在栈上的某个偏移处构造 `lab3Class` 对象，对应 `obj0(3,7)`；
- 第二次以 `(2,8)` 为参数在另一个偏移处构造对象，对应 `obj1(2,8)`；
这与 C++ 源码中的 `lab3Class obj0(3, 7); lab3Class obj1(2, 8);` 完全一致。

- **成员函数调用与返回值：**

对象构造完成后，汇编代码将第一个对象地址放入 `x0`（作为 `this`），第二个对象地址放入 `x1`，然后调用

`_ZN9lab3Class4funcES_`。在该函数内部，通过 `getM/getN` 获取 `other` 的成员，再配合当前对象的 `mPlusN2_`，调用 `_Z3maxii` 和 `_Z3minRiS_` 计算最大值和最小值，并返回它们的差值。
`lab3Task3_23307130447` 只需把这个返回值原样放在 `w0` 中作为自己的返回结果。

- **与 main 的关系：**

在 `main` 的汇编段中，可以看到对 `_Z21lab3Task3_23307130447v` 的一次调用，随后把返回的 `w0` 作为参数传给 `printf/__printf_chk`，打印出 `"ID=23307130447 Lab3 Task3 = 0"`。也就是说，`lab3Task3_23307130447` 在整个程序中的角色就是“组织两个对象并调用 `func` 得到 `task3_value`”，而最终的输出则由 `main` 负责格式化打印。

从这一入口函数的视角看，Task3 的汇编结构与前文还原的 C++ 代码逐行对应：`lab3Task3_23307130447` 负责对象生命周期和调用顺序，`lab3Class::func` 负责具体数值计算，这也进一步验证了 `task3_value = 0` 的结论是正确的。

尝试还原整个代码

// Task3 中涉及到的 `max/min` 以及类和调用结构，可以还原为下面的 C++ 代码：

```
int max(int a, int b) {
    // 与 Task1 中的 max 一样，返回较大的那个，只是实现更精简
    return (a > b) ? a : b;
}

int min(const int &a, const int &b) {
    // 通过引用传参，返回较小的那个
    return (a < b) ? a : b;
}

class lab3Class {
public:
    lab3Class(int m, int n) : m_(m), n_(n) {
        // 构造函数中计算并保存 2*(m + n)
        mPlusN2_ = 2 * (m_ + n_);
    }

    int getM() const { return m_; }
    int getN() const { return n_; }

    int func(const lab3Class &other) const {
        // 先对另一个对象的 m、n 求和并乘以 2
        int tmp = 2 * (other.getM() + other.getN());
        // a 是这两个值中的较大者，b 是较小者
        int a = max(tmp, mPlusN2_);
        int b = min(tmp, mPlusN2_);
        // 返回差值
        return a - b;
    }

private:
    int m_;
    int n_;
    int mPlusN2_;
};
```

```

int lab3Task3_23307130447() {
    lab3Class obj0(3, 7);
    lab3Class obj1(2, 8);

    int task3_value = obj0.func(obj1);
    return task3_value;
}

int main(int argc, const char** argv) {
    // 实际运行时打印出的 task3_value 为 0
    printf("ID=23307130447 Lab3 Task3 = %d\n", lab3Task3_23307130447());
    return 0;
}

```

值

task3_value = 0

解释该汇编代码中max、min的实现上与Task1的差异

在 Task3 中，`_Z3maxii` 与 `_Z3minRiS_` 的汇编实现，和 Task1 中的 `max` 在“功能上相同、实现方式更简洁”：

- Task1 的 `max`：
 - 先把两个参数分别压栈，再把其中一个先假设为结果，
 - 然后比较后再按条件把另一个覆盖回去，最后从栈上取回结果；
 - 逻辑更接近“通过一个中间变量 `res` 来保存当前最大值”的写法。
- Task3 的 `max(int a, int b)`：
 - 汇编里只是把两个参数存在栈上，比较后直接从对应位置加载，
 - 没有额外的中间变量，也没有多余的写回操作，控制流更短、更接近三目运算符 `a > b ? a : b`。
- Task3 的 `min(const int &a, const int &b)`：
 - 通过引用（指针）传参，先解引用再比较，
 - 但本质上仍然是根据比较结果从两个地址中选出一个值返回。
 - 与 Task1 不同的是，这里在汇编层面可以看到 **多了一次通过指针访问内存** 的过程，对应 C++ 中的引用语义。

另外，Task3 里 `max/min` 被封装成独立函数，并在 `lab3Class::func` 中被多次复用；

而 Task1 中的 `max` 更像是为那段简单逻辑“单独写的一个小函数”，没有被其他代码（如 `func1/func2`）再抽象利用。

max、min的源码

```
int max(int a, int b) {
    return (a > b) ? a : b;
}

int min(const int &a, const int &b) {
    return (a < b) ? a : b;
}
```

实验所有运行截图

我把拼凑的代码（armv8汇编和C/C++）全部做了区分+汇编合并，最终得到完整的可执行文件。以此来验证我的分析。

```
sudo apt update
sudo apt install g++-aarch64-linux-gnu libstdc++-13-dev-arm64-cross

aarch64-linux-gnu-g++ Lab3_Task1_2_23307130447.S -o task1_2_aarch64
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task1_2_aarch64

aarch64-linux-gnu-g++ Lab3_Task3_23307130447.S -o task3_aarch64
qemu-aarch64 -L /usr/aarch64-linux-gnu ./task3_aarch64
```

Task1&Task2

```
# pore @ pore in ~/pore26/pore_23307130447/Lab3 on git:master x [13:19:56] C:1
● $ aarch64-linux-gnu-g++ Lab3_Task1_2_23307130447.S -o task1_2_aarch64

# pore @ pore in ~/pore26/pore_23307130447/Lab3 on git:master x [13:21:26]
● $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task1_2_aarch64
ID=23307130447 Lab3 Task1 = 81
ID=23307130447 Lab3 Task2 = ,.000112233334445677789Tabcdeeeefghhijklmnoooopqrrstuvwxyz
```

Task3

```
# pore @ pore in ~/pore26/pore_23307130447/Lab3 on git:master x [13:38:57]
● $ aarch64-linux-gnu-g++ Lab3_Task3_23307130447.S -o task3_aarch64

# pore @ pore in ~/pore26/pore_23307130447/Lab3 on git:master x [13:39:00]
● $ qemu-aarch64 -L /usr/aarch64-linux-gnu ./task3_aarch64
ID=23307130447 Lab3 Task3 = 0
```